

Dashboard de Operaciones y Logística

Pipeline de datos con arquitectura Medallion (Bronze · Silver · Gold) y dashboard web en Flask

Proyecto	Dashboard de Operaciones y Logística (carpeta <code>light</code>)
Tipo	Proyecto de Data Engineering con datos simulados
Dominio	Transporte de pasajeros y paquetería entre seis ciudades
Tecnologías	Python · Pandas · Flask · Plotly · Jinja2 · HTML/CSS/JavaScript
Base del análisis	Código fuente (<code>Silver.py</code> , <code>gold.py</code> , <code>app.py</code> , <code>index.html</code>) y archivos CSV de cada capa
Versión / fecha	1.0 · 27 de septiembre de 2026



Datos crudos → datos limpios → datos analíticos → visualización

Contenido del documento

N.º	Sección	Qué encontrarás
1	Resumen ejecutivo	Qué se construyó y resultados clave del periodo analizado.
2	Alcance y tecnologías	Stack utilizado y dependencias.
3	Estructura del proyecto	Carpetas, scripts y responsabilidades.
4	Arquitectura	Diagrama por capas, principios de diseño y flujo de ejecución.
5–7	Capas Bronze, Silver y Gold	Qué contiene cada capa y qué se hizo en ella.
8	Dashboard con Flask y HTML	Backend, frontend e interactividad.
9	Data lineage	Linaje a nivel de tabla, de campo y de indicador; llaves y relaciones.
10	Calidad de datos	Validaciones de integridad y conciliación realizadas.

1. Resumen ejecutivo

Este proyecto implementa un **pipeline de datos** y un **dashboard analítico** para una operación de transporte de pasajeros y paquetería que conecta seis ciudades: Guadalajara, Zapopan, Puerto Vallarta, Bucerías, Nuevo Vallarta y Tepic. Los datos son simulados y el proyecto tiene fines demostrativos.

El pipeline sigue la **arquitectura Medallion**: los datos crudos se conservan en **Bronze**, se limpian y estandarizan en **Silver**, y se transforman en indicadores de negocio en **Gold**. Un dashboard web desarrollado con **Flask y HTML** consume únicamente la capa Gold.

Qué se hizo

- Se organizó el proyecto en tres capas de datos, cada una en su propia carpeta y con archivos CSV como formato de intercambio.
- Se desarrollaron dos procesos ETL en Python (`Silver.py` y `gold.py`), idempotentes y con registro de ejecución (logs) por capa.
- Se construyó una **tabla maestra por viaje** que integra telemetría GPS, abordajes, ventas de pasajeros y de paquetería, con indicadores de rentabilidad y eficiencia.
- Se generaron agregados por unidad, por ruta, por chofer (horas de servicio semanales) y un ranking de los 10 viajes con mayor ocupación.
- Se creó un dashboard con KPIs generales, filtros por trayecto y unidad, gráficas interactivas y una tabla de los viajes más ocupados.

2. Alcance y tecnologías

El alcance cubre desde los archivos crudos de la capa Bronze hasta la visualización en navegador. Las dependencias están declaradas en `requirements.txt`.

Componente	Tecnología / versión	Uso en el proyecto
Lenguaje	Python 3	Procesos ETL, backend del dashboard.
Procesamiento de datos	pandas ≥ 2.0.0	Lectura de CSV, limpieza, joins, agregaciones y cálculo de KPIs.
Cómputo numérico	numpy ≥ 1.24.0	Declarado en requirements.txt; los scripts no lo importan directamente (es dependencia de Pandas).
Servidor web	Flask ≥ 3.0.0	Ruta / que prepara los datos y renderiza el dashboard.
Gráficas	plotly ≥ 5.18.0	Gráficas de barras interactivas (Plotly Express).
Plantillas	Jinja2 ≥ 3.1.2	Plantilla index.html con ciclos y variables enviadas desde Flask.
Interfaz	HTML, CSS y JavaScript	Diseño con CSS Grid, filtros dinámicos en JavaScript nativo.
Almacenamiento	Archivos CSV	Formato de cada capa (Bronze, Silver y Gold).
Registro	Módulo logging	Logs por ejecución en logs/silver/ y logs/gold/.

3. Estructura del proyecto

La carpeta del proyecto se organiza por capas de datos y por componentes de aplicación:

```

Main/
├── Bronze/           6 CSV con datos crudos
├── Silver/           4 CSV con datos limpios
├── Gold/             5 CSV con datos analíticos (KPIs)
├── templates/
│   └── index.html    Plantilla del dashboard (Jinja2 + CSS + JS)
├── Silver.py         ETL Bronze → Silver
├── gold.py           ETL Silver → Gold
├── app.py            Aplicación Flask (dashboard)
├── requirements.txt  Dependencias
└── logs/             Se genera al ejecutar (silver/ y gold/)

```

Responsabilidad de cada componente

Componente	Rol	Entrada	Salida
Silver.py	ETL de Bronze a Silver: limpieza, estandarización y enriquecimiento.	Bronze/ (5 de 6 archivos)	Silver/ (4 CSV) y logs/silver/
gold.py	ETL de Silver a Gold: integración, KPIs y agregaciones.	Silver/ (4 CSV)	Gold/ (5 CSV) y logs/gold/

Componente	Rol	Entrada	Salida
app.py	Backend Flask: prepara datos, KPIs y gráficas.	Gold/ (4 de 5 archivos)	Variables para la plantilla
templates/index.html	Interfaz del dashboard con filtros dinámicos.	Variables de Flask	Página HTML en el navegador
requirements.txt	Lista de dependencias de Python.	—	—

4. Arquitectura

La solución se organiza en capas: cada proceso lee solo de la capa anterior y escribe en la siguiente. El dashboard queda desacoplado del procesamiento porque solo consume Gold.

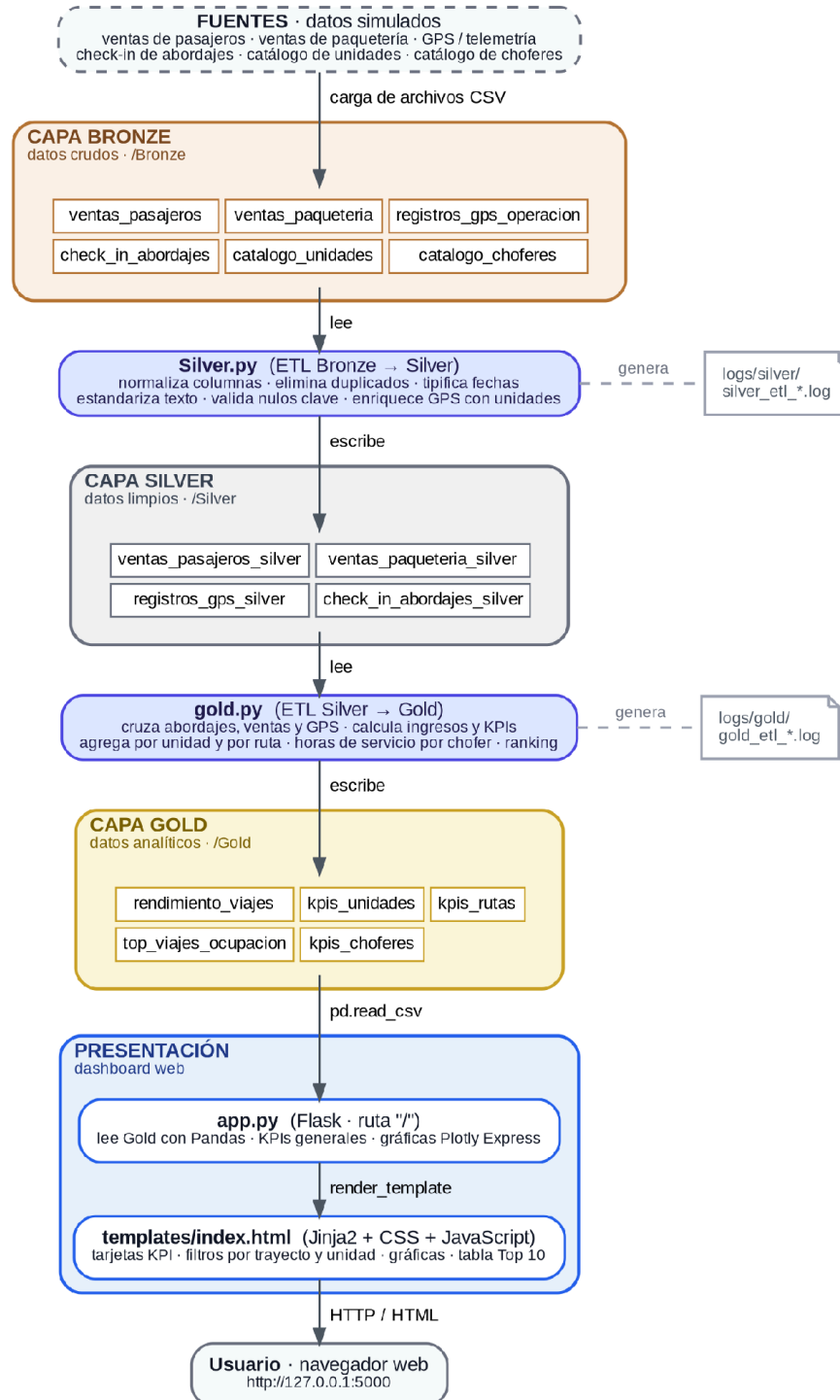


Figura 1. Arquitectura por capas del proyecto

4.1 Principios de diseño

Principio	Cómo se aplica
Separación por capas (Medallion)	Bronze conserva el dato original; Silver limpia y estandariza; Gold calcula indicadores. Cada capa depende solo de la anterior.
Idempotencia	Cada ejecución sobrescribe los CSV de salida de su capa (to_csv), por lo que ejecutar el proceso varias veces produce el mismo resultado.
Trazabilidad	Cada ejecución crea un log con fecha y hora (silver_etl_AAAAMMDD_HHMMSS.log, gold_etl_...log) con mensajes de avance y errores.
Manejo de errores	La lectura de archivos está protegida con try/except: se registra el error y se relanza. En Flask, si falta un archivo Gold se muestra un mensaje con la ruta buscada.
Presentación desacoplada	El dashboard no lee Bronze ni Silver. Solo realiza cálculos menores de presentación (nombre de ruta, consumo promedio por viaje, KPIs generales).

4.2 Flujo de ejecución

El orden de ejecución es manual y secuencial, ya que el proyecto no incluye un orquestador:

1. ``Silver.py`` lee Bronze y genera los 4 archivos de Silver.
2. ``gold.py`` lee Silver y genera los 5 archivos de Gold.
3. ``app.py`` lee Gold y publica el dashboard en `http://127.0.0.1:5000`.

5. Capa Bronze: datos crudos

Bronze almacena los datos **tal como llegan de la fuente**, sin transformaciones. Está formada por seis archivos CSV: cuatro de transacciones y dos catálogos. Estos archivos conservan las inconsistencias propias del dato original.

Archivo	Filas	Cols	Contenido y llave
ventas_pasajeros_bronze.csv	500	8	Venta de boletos: pasajero, fecha de nacimiento, género, origen, destino, fecha de salida e importe. Llave: <code>id_pasajero</code> .
ventas_paqueteria_bronze.csv	200	6	Venta de envíos: fecha, origen, destino, peso e importe. Llave: <code>id Paquete</code> .
registros_gps_operacion_bronze.csv	464	9	Telemetría: una fila por viaje con unidad, chofer, salida, llegada, kilómetros y combustible. Llave: <code>id viaje</code> .
check_in_abordajes_bronze.csv	700	8	Tabla puente. Cada fila indica un pasajero (500) o un paquete (200) a bordo de un viaje.
catalogo_unidades_bronze.csv	20	4	Placa, serie y fecha de mantenimiento por unidad. Llave: <code>id unidad</code> .
catalogo_choferes_bronze.csv	25	3	Nombre y fecha de ingreso por chofer. Llave: <code>id chofer</code> .

6. Capa Silver: datos limpios y estandarizados

6.1 Qué hace Silver.py

`Silver.py` lee cinco de los seis archivos de Bronze (el catálogo de choferes no se utiliza), aplica reglas de limpieza y estandarización, y escribe **cuatro archivos** en la carpeta `Silver/`. El catálogo de unidades no genera un archivo propio: se integra en el dataset de GPS.

6.2 Transformaciones por dataset

Dataset	Transformaciones aplicadas	Filas (ent. → sal.)
Ventas de pasajeros	Nombres de columnas en minúsculas y sin espacios. Eliminación de duplicados por <code>id_pasajero</code> . <code>fecha_salida</code> convertida a fecha y hora. <code>origen</code> y <code>destino</code> en MAYÚSCULAS y sin espacios extremos. Se descartan filas sin <code>id_pasajero</code> o sin <code>total_venta</code> .	500 → 500
Ventas de paquetería	Normalización de columnas y eliminación de duplicados por <code>id_paquete</code> . Se renombra <code>total_venta</code> a <code>ingreso_paqueteria</code> . <code>fecha</code> convertida a fecha y hora; <code>origen</code> y <code>destino</code> en MAYÚSCULAS. Se descartan filas sin <code>id_paquete</code> o sin <code>ingreso</code> .	200 → 200
Registros GPS + unidades	Normalización de columnas; deduplicación del catálogo por <code>id_unidad</code> y del GPS por <code>id_viaje</code> . Fechas de partida y llegada en formato AAAA-MM-DD. Enriquecimiento con unión izquierda por <code>id_unidad</code> : se agregan <code>placa</code> , <code>serie</code> y <code>timestamp_mantenimiento</code> . Se seleccionan 12 columnas y se conserva <code>id_chofer</code> para los KPIs de Gold.	464 → 464
Check-in / abordajes	Normalización de columnas. Eliminación de duplicados por (<code>id_viaje</code> , <code>id_pasajero</code> , <code>id_paqueteria</code>). <code>fecha_partida</code> en formato AAAA-MM-DD; <code>origen</code> y <code>destino</code> en MAYÚSCULAS.	700 → 700

Resultado. Los cuatro datasets conservaron el 100 % de sus registros. Los controles de duplicados y nulos están implementados.

6.3 Registro de ejecución

`Silver.py` crea `logs/silver/silver_etl_AAAAMMDD_HHMMSS.log` y escribe los mensajes tanto en el archivo como en consola (nivel INFO). Se registra el inicio, la lectura de Bronze, el número de filas de entrada de cada dataset y el cierre exitoso. Si falla la lectura, se registra un mensaje de error y se relanza la excepción.

7. Capa Gold: datos analíticos

`gold.py` lee cuatro archivos de Silver (GPS, abordajes, ventas de pasajeros y ventas de paquetería) y produce **cinco tablas** listas para consumo analítico. Genera sus propios logs en `logs/gold/`.

7.1 Productos de la capa

Archivo	Grano	Filas	Contenido
rendimiento_viajes_gold.csv	1 fila por viaje	464	Tabla maestra de 22 columnas: datos operativos, pasajeros, paquetes, ingresos e indicadores por viaje.
kpis_unidades_gold.csv	1 fila por unidad	20	Viajes, kilómetros, combustible, pasajeros, ingresos y eficiencia.
kpis_rutas_gold.csv	1 fila por origen-destino	30	Viajes, pasajeros, paquetes, ingresos y promedios; ordenada por ingreso.
top_viajes_ocupacion_gold.csv	10 viajes	10	Los 10 viajes con más pasajeros.
kpis_choferes_gold.csv	Chofer × semana ISO	114	Horas manejadas, viajes y kilómetros por chofer y semana.

7.2 Construcción de la tabla maestra

1. **Conteos por viaje:** pasajeros (filas de abordaje con `id_pasajero`) y paquetes (filas con `id_paqueteria`).
2. **Ingresos por viaje:** se une abordajes con ventas de pasajeros por `id_pasajero` y con ventas de paquetería por `id_paqueteria = id_paquete`, y se suma el importe por viaje.
3. **Ruta del viaje:** origen y destino se toman del primer registro de abordaje de cada viaje.
4. **Base:** se parte de `registros_gps_silver` y se le unen (unión izquierda por `id_viaje`) la ruta y las cuatro métricas anteriores; los nulos se llenan con 0.
5. **Indicadores derivados:** ingreso total, km por litro, ingreso por km y yield por pasajero-km.

7.3 KPIs y fórmulas

Indicador	Fórmula	Nivel
ingreso_total	<code>ingresos_pasajeros + ingresos_paqueteria</code>	Viaje
km_por_litro	<code>kms_recorridos ÷ consumo_combustible (2 decimales)</code>	Viaje
ingreso_por_km	<code>ingreso_total ÷ kms_recorridos (si km = 0 se divide entre 1)</code>	Viaje
yield_pasajero_km	<code>ingreso_total ÷ (total_pasajeros × kms_recorridos); 0 si no hay pasajeros o kilómetros</code>	Viaje
eficiencia_promedio_km_l	<code>kms_totales ÷ combustible_total (promedio ponderado)</code>	Unidad
ingreso_promedio_por_km	<code>ingresos_totales ÷ kms_totales</code>	Unidad
ingreso_promedio_por_km_ruta	<code>ingreso_total_acumulado ÷ (viajes_realizados × kms_promedio)</code>	Ruta
total_horas_manejadas	Suma de (llegada – partida) en horas, por chofer y semana ISO	Chofer / semana

7.4 Horas de servicio por chofer

A partir de las fechas y horas de partida y llegada se calcula la duración de cada viaje (entre 2.1 y 4.6 horas, con promedio de 3.3 h). Luego se agrupa por chofer y semana ISO del año. El resultado son **114 registros chofer-semana** que cubren las semanas 36 a 40; el máximo observado es de **33.56 horas** en una semana.

8. Dashboard con Flask y HTML

8.1 Backend (app.py)

La aplicación Flask expone una única ruta, `/`, cuya función `dashboard()` sigue estos pasos:

1. **Carga de datos:** lee cuatro archivos de Gold con Pandas (`rendimiento_viajes`, `kpis_unidades`, `kpis_rutas`, `top_viajes_ocupacion`). Si alguno falla, devuelve un mensaje de error con la carpeta buscada.
2. **Preparación de rutas:** crea la columna `ruta` ("ORIGEN → DESTINO"), ordena por ingreso y convierte los datos a registros para enviarlos a JavaScript.
3. **Preparación de unidades:** calcula `consumo_promedio_por_viaje` (combustible total ÷ viajes) y ordena de mayor a menor consumo.
4. **KPIs generales:** ingresos totales, pasajeros, kilómetros y eficiencia global (km ÷ litros), formateados como texto.
5. **Gráficas:** dos gráficas de barras con Plotly Express (top 5 de rutas por ingreso acumulado y eficiencia km/L por unidad), convertidas a HTML con `pio.to_html`.
6. **Renderizado:** envía KPIs, gráficas, tabla Top 10 y listas de rutas y unidades a `index.html` con `render_template`.

9. Data lineage

El linaje muestra el recorrido de los datos desde los archivos de Bronze hasta cada elemento visible del dashboard. Los nodos con línea punteada son archivos que **no se consumen** en la etapa siguiente.

9.1 Linaje a nivel de tabla

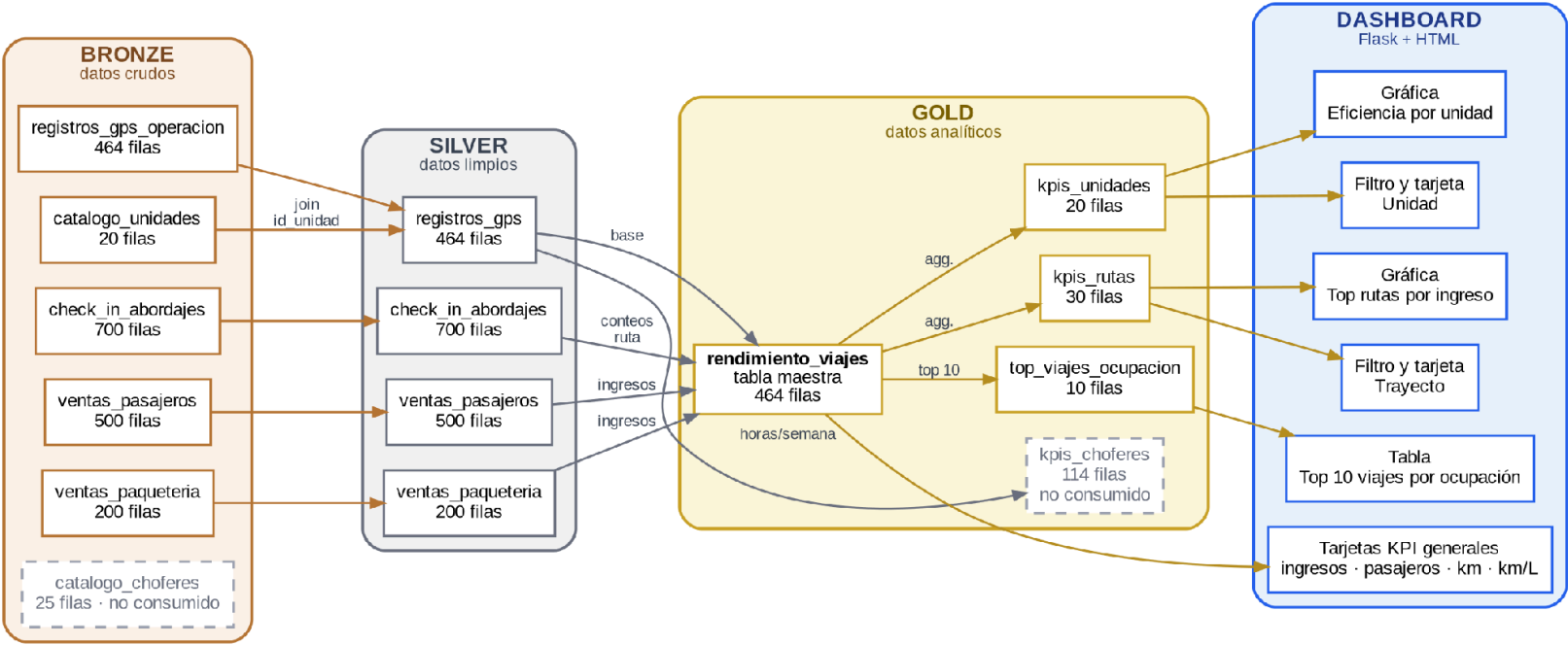


Figura 2. Linaje de datos: Bronze → Silver → Gold → Dashboard (las etiquetas indican la operación aplicada)

Transición	Qué ocurre	Ejemplo
Bronze → Silver	Limpieza y estandarización; el GPS se enriquece con el catálogo de unidades.	catalogo_unidades + registros_gps
Silver → Gold	Integración en una tabla maestra por viaje y agregación por unidad, ruta y chofer.	abordajes + ventas → ingresos_pasajeros
Gold → Dashboard	Cada componente lee un archivo Gold; kpis_choferes aún no tiene vista.	kpis_rutas → gráfica “Ingresos por Ruta”

9.2 Linaje de campos de la tabla maestra

La siguiente tabla detalla de dónde proviene cada campo de `rendimiento_viajes_gold`. Los nombres de archivo se muestran sin el sufijo de capa (`_bronze`, `_silver`, `_gold`).

Campo(s) en Gold	Origen en Silver	Origen en Bronze	Transformación
<code>id_viaje</code> , <code>id_unidad</code> , <code>id_chofer</code> , <code>kms_recorridos</code> , <code>consumo_combustible</code> , <code>hora_partida</code> , <code>hora_llegada</code>	<code>registros_gps</code>	<code>registros_gps_operacion</code>	Sin cambios.
<code>fecha_partida</code> , <code>fecha_llegada</code>	<code>registros_gps</code>	<code>registros_gps_operacion</code>	Convertidas a texto AAAA-MM-DD.
<code>placa</code> , <code>serie</code> , <code>timestamp_mantenimiento</code>	<code>registros_gps</code>	<code>catalogo_unidades</code>	Unión izquierda por <code>id_unidad</code> .
<code>origen</code> , <code>destino</code>	<code>check_in_abordajes</code>	<code>check_in_abordajes</code>	MAYÚSCULAS; primer registro por viaje.
<code>total_pasajeros</code>	<code>check_in_abordajes</code>	<code>check_in_abordajes</code>	Conteo de filas con <code>id_pasajero</code> por viaje.
<code>total_paquetes</code>	<code>check_in_abordajes</code>	<code>check_in_abordajes</code>	Conteo de filas con <code>id_paqueteria</code> por viaje.
<code>ingresos_pasajeros</code>	<code>check_in_abordajes</code> + <code>ventas_pasajeros</code>	<code>check_in_abordajes</code> + <code>ventas_pasajeros</code>	Unión por <code>id_pasajero</code> ; suma de <code>total_venta</code> por viaje.
<code>ingresos_paqueteria</code>	<code>check_in_abordajes</code> + <code>ventas_paqueteria</code>	<code>check_in_abordajes</code> + <code>ventas_paqueteria</code>	Unión por <code>id_paqueteria</code> = <code>id Paquete</code> ; suma de <code>ingreso_paqueteria</code> (antes <code>total_venta</code>).
<code>ingreso_total</code> , <code>km_por_litro</code> , <code>ingreso_por_km</code> , <code>yield_pasajero_km</code>	Calculados en Gold	—	Fórmulas de la sección 7.3.

9.3 Linaje de los indicadores del dashboard

Elemento del dashboard	Campos utilizados	Archivo Gold	Origen final (Bronze)
Ingresos totales	<code>ingreso_total</code> (suma)	<code>rendimiento_viajes</code>	<code>ventas_pasajeros</code> , <code>ventas_paqueteria</code> (vía <code>check_in_abordajes</code>)
Pasajeros totales	<code>total_pasajeros</code> (suma)	<code>rendimiento_viajes</code>	<code>check_in_abordajes</code>
Kilómetros recorridos	<code>kms_recorridos</code> (suma)	<code>rendimiento_viajes</code>	<code>registros_gps_operacion</code>
Eficiencia global	$\Sigma \text{kms_recorridos} \div \Sigma \text{consumo_combustible}$	<code>rendimiento_viajes</code>	<code>registros_gps_operacion</code>

Elemento del dashboard	Campos utilizados	Archivo Gold	Origen final (Bronze)
Gráfica “Ingresos por Ruta”	origen, destino, ingreso_total_acumulado	kpis_rutas	Ventas + check_in_abordajes + GPS
Tarjeta de trayecto	viajes_realizados, ingreso_total_acumulado, combustible_promedio, ingreso_promedio_por_km_ruta	kpis_rutas	Ventas + check_in_abordajes + GPS
Gráfica “Eficiencia por Unidad”	id_unidad, eficiencia_promedio_km_l	kpis_unidades	registros_gps_operacion, catalogo_unidades
Tarjeta de unidad	combustible_total, total_viajes, eficiencia_promedio_km_l; el consumo por viaje se calcula en app.py	kpis_unidades	registros_gps_operacion, catalogo_unidades
Tabla Top 10	id_viaje, id_unidad, origen, destino, total_pasajeros, ingreso_total, km_por_litro	top_viajes_ocupacion	Abordajes + ventas + GPS

9.4 Llaves y relaciones

Llave	Dónde aparece	Relación observada
id_viaje	registros_gps (una fila por viaje), check_in_abordajes	1 : N. Cada viaje tiene de 1 a 5 registros de abordaje.
id_pasajero	ventas_pasajeros, check_in_abordajes	1 : 1. Cada pasajero vendido aparece una sola vez en abordajes.
id_paquete / id_paqueteria	ventas_paqueteria, check_in_abordajes	1 : 1. El campo se llama distinto en cada tabla.
id_unidad	catalogo_unidades, registros_gps, check_in_abordajes	1 : N. Una unidad realiza varios viajes.
id_chofer	catalogo_choferes, registros_gps	1 : N. Se usa solo el identificador; el catálogo no se une.

10. Calidad de datos y validaciones

Además de las reglas implementadas en `Silver.py`, se ejecutaron validaciones sobre los archivos entregados para comprobar la integridad entre capas:

Validación	Resultado
Registros conservados de Bronze a Silver	100 % en los cuatro datasets (500, 200, 464 y 700 filas).
Viajes de GPS con registros de abordaje, y viceversa	464 de 464 en ambos sentidos; sin viajes huérfanos.
Pasajeros y paquetes de abordajes con venta registrada	100 %; sin identificadores huérfanos.

Validación	Resultado
Ventas sin abordaje asociado	Ninguna: 500 pasajeros y 200 paquetes aparecen en abordajes.
Choferes y unidades del GPS presentes en catálogos	25 de 25 choferes y 20 de 20 unidades.
Filas de abordaje con pasajero y paquete a la vez, o con ninguno	0 casos: cada fila corresponde a uno solo.
Conciliación de ingresos (Silver vs. Gold)	Coincide al centavo: pasajeros \$323,596.87 y paquetería \$84,952.06.
Viajes con ingreso igual a cero en Gold	0 de 464.
Duraciones de viaje negativas	0; rango de 2.1 a 4.6 horas.

